

Data Center Construction Guide & EPC Intelligence System

Problem:

There are two key intelligence challenges around data center projects:

- Prior to construction, there is a need to make a decision about the viability of the site considering terrain, water, agriculture, power, connectivity, hazards, civil complexity, and regulatory issues.
- During construction, there are many design and equipment changes and to understand one change, a manual process of connecting project specification, drawings, equipment schedules, equipment manufacturer data, site information and code is necessary.

The common underlying issue is fragmentation. No single source holds all the answers: project specification knows what is required, equipment manufacturer data know what can be done with an equipment model and Mireye geospatial system knows what is true about the physical site itself.

Our platform connects these pieces and lets an agent decide what evidence is actually needed for the specific case.

Our Solution:

A **multi-agent AI** platform that evaluates data-center sites **before construction and intelligently manages engineering changes during construction**. Specialized agents combine Mireye's physical-world intelligence, project documents, manufacturer data, and external datasets to assess **site feasibility, water/power/civil risks, equipment changes, their downstream impacts, and missing evidence**. A supervisor agent orchestrates these investigations and produces an **evidence-backed recommendation** with the required next action.

When a user enters the platform, they choose one of two workflows: **Before Construction** or **During Construction**.

Both workflows share the same project intelligence, including **Mireye and external datasets, project documents, manufacturer information, advanced RAG, a knowledge graph, evidence tracking, deterministic engineering calculations, and an agentic orchestration layer**.

Before Construction	During Construction
Explore sites and select the best one based on water, power, terrain, connectivity, hazards, environmental impact, regulations, and community needs.	Know the implications of equipment or design changes before approving or installing them.

The two workflows are connected because the physical site becomes part of the engineering context. A project document can describe a requirement, a manufacturer document can describe equipment capability, and Mireye can provide site-dependent facts needed to determine whether that equipment is appropriate for the actual project.

Features:

1. Site Intelligence (Before Construction)

Evaluates candidate sites across geography, water, power, risks, agriculture, civil complexity, and regulations. **Scores and explains major risks** and suggests **lower-risk alternatives** with similar infrastructure potential.

2. Equipment Change & Delta Analysis

Identifies **replaced equipment**, verifies **comparability**, and calculates true deltas in weight, load, dimensions, electrical demand, refrigerant use, and operating conditions. Flags potential **structural coordination** without claiming structural adequacy.

3. Dynamic Investigation & Impact Tracing

Builds a **situation-specific investigation plan** instead of a fixed checklist and traces impacts across the **project dependency graph**. Changes such as a chiller replacement can trigger structural, electrical, cooling, controls, installation, and commissioning reviews.

4. Evidence Extraction: Documents, Manufacturers & Mireye

Extracts **requirements** from specifications, schedules, submittals, and drawings, capability data from **manufacturer datasheets**, and physical-world facts from **Mireye on demand**. Every fact is **source-traceable**, with only **relevant fields** retrieved for an **auditable reasoning path**.

5. Mireye Evidence & Assumption Integrity

Tracks source, time, location, and status for every observation, explicitly marking **missing data**. Compares **manufacturer evidence with site conditions**, identifies **evidence gaps**, and marks **assumptions stale** when conditions change.

6. Verification & Next Action

Validates model identity, values, units, configuration, rating conditions, sources, and location before accepting comparisons. Converts findings into **RFIs, vendor evidence requests, or review comments**, classified as **FIRST-PASS CHECKS CLOSED, NEEDS INFORMATION, or ENGINEER REVIEW**.

Tech Stack:

- **Frontend:** React, TypeScript, Vite, Tailwind CSS, shadcn/ui, MapLibre/Leaflet, Recharts & Lucide.
- **Backend:** Python, FastAPI, Pydantic, Uvicorn with LangGraph orchestrating the Understand → Plan → Evidence → Signals → Replan → Impact → Action workflow.
- **AI/RAG:** LLMs for document extraction, investigation, reasoning, explanations & RFI drafting; PyMuPDF/Docling + PostgreSQL/pgvector for RAG.
- **Knowledge & Engineering:** Neo4j Knowledge Graph for relationship/impact tracing, while Python + Pint handles unit conversion, scoring, thresholds & engineering checks—not the LLM.
- **Infrastructure:** Supabase PostgreSQL/Storage, Neo4j Aura, with Vercel for frontend and Render/Railway for backend deployment.

Data & Dataset Requirements:

The system should involve multiple data types. Mireye would represent the real-world/geographical layer.

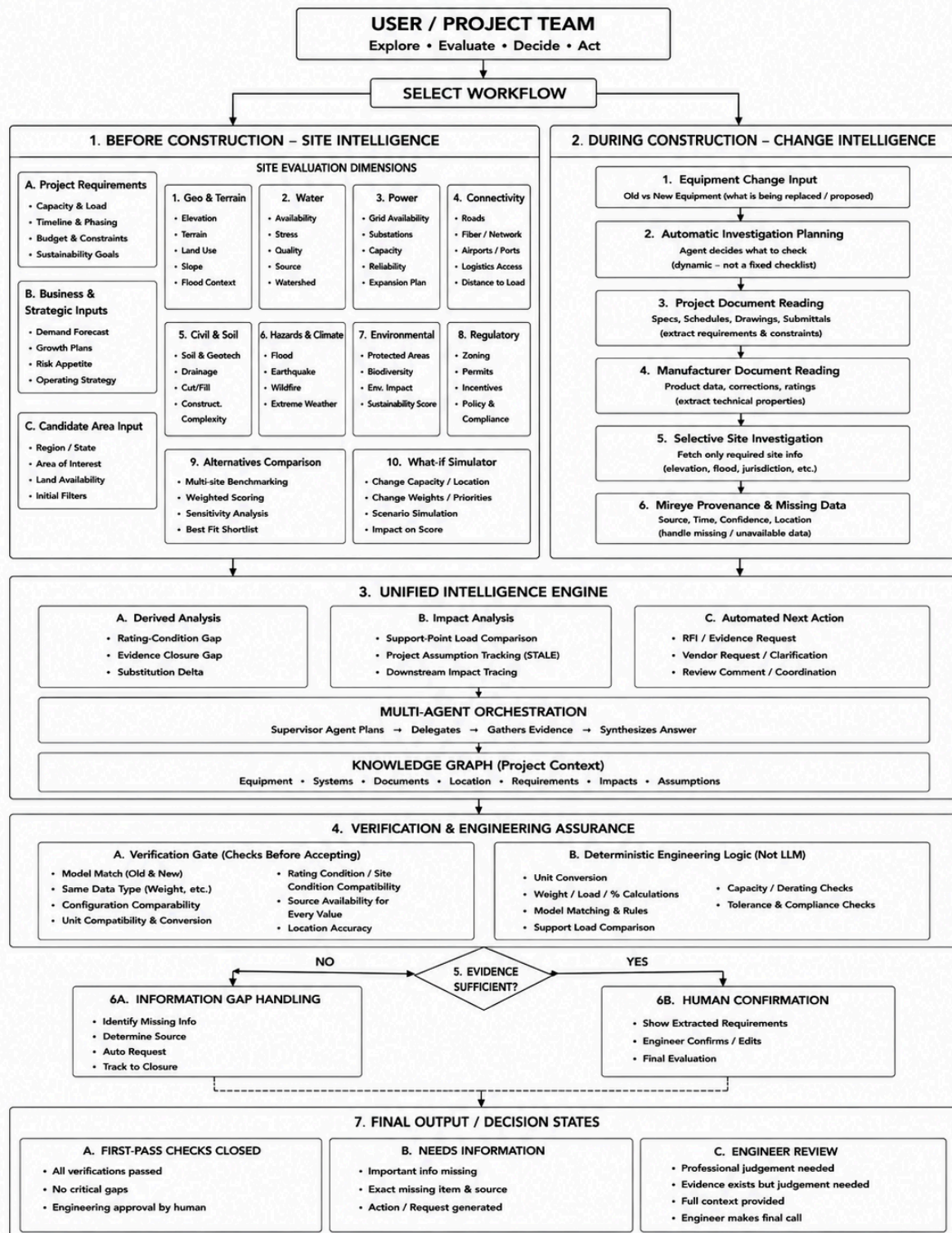
Dataset / Source	What we use it for
Mireye	Terrain, elevation, soils, water, flood/wildfire risk, buildings, roads, utility/grid context, parcels, physical site facts + provenance
Project specifications	Required capacity, dimensions, conditions, tolerances, code clauses
Equipment schedules/drawings	Existing equipment, location, connections, design assumptions
Submittals	Proposed equipment/model and submitted evidence
Manufacturer PDFs	Performance, weight, dimensions, support loads, electrical values, correction rules
AHRI Directory	Independent certified equipment-performance evidence where applicable
Project/code documents	Governing code requirements and design criteria
Neo4j project graph	Equipment → assumption → discipline → activity → commissioning dependencies

External datasets:

- USGS: water/elevation/geological context
- FEMA NFHL: flood information
- USDA: agriculture/cropland
- NOAA: climate/weather design context
- EIA / utility sources: electric-grid context
- FCC broadband data: connectivity context
- Manufacturer data: equipment rules and capabilities
- AHRI: equipment certification
- Project documents: actual engineering requirements

Architecture Flow:

The architecture is designed around one principle: the agent decides what needs to be investigated, while deterministic software performs important calculations and verification. This separates flexible AI investigation from safety-critical numerical logic.



Mireye API Calls:

- **GET /v1/meta/fields**
Fetch available Mireye fields and cache them. Prevents hard-coding or guessing field names.
- **POST /v1/geocode**
Converts an address into coordinates and location-resolution information.
- **POST /v1/fetch**
Main endpoint for our platform. The agent fetches only the exact physical-world fields needed, such as elevation, flood or soil data.
Flow:
Equipment/site question → Agent decides required fields → **/fetch**
- **POST /v1/ask**
Used for natural-language physical-world questions and exploratory investigation.
Use /ask for: flexible investigation and unusual questions.
Use /fetch for: calculations, site ranking and repeatable checks.
- **POST /v1/ask/stream**
Streaming version of **/ask**, useful for showing live agent investigation in the UI.
- **Site APIs**
 - **POST /v1/sites**
 - **GET /v1/sites/{site_id}**
 - **POST /v1/ask-site**
- Useful for storing and investigating candidate sites in the **Before Construction** workflow.
- **POST /v1/feature-requests**
If a required field is unavailable, the agent records the gap instead of hallucinating a value.
- **Mireye MCP**
Supports agent tools such as **mireye_ask**, **mireye_fetch**, and **mireye_geocode**. REST API is sufficient for our MVP.

Two-Week Implementation Plan:

Days 1–2: Architecture & Data

- Freeze both workflows and shared schemas.
- Save Mireye field catalog.
- Collect project, Carrier, AHRI, and site data.
- Verify all demo values.

Days 3–4: RAG & Extraction

- Build PDF parsing → chunking → embeddings → pgvector.
- Extract requirements, values, units, conditions, equipment tags, and source spans.
- Add requirement confirmation UI.

Day 5: Mireye Integration

- Build **MireyeTool**: catalog, geocode, fetch, ask.
- Store source, timestamp, confidence, and status.
- Add cache/fallback.

Days 6–7: Before Construction

- Evaluate Terrain, Water, Hazards, Soil/Civil, Power, and Environment.
- Rank candidate sites with explainable scores.
- Use progressive investigation: many sites → shortlist → deeper analysis.

Days 8–10: During Construction

- Compare old vs new equipment.
- Calculate weight, support-load, dimension, refrigerant, and electrical deltas.
- Add **CLOSED / OPEN / TRIGGERED / SKIPPED**.
- Build Verification Gate for model, configuration, units, conditions, sources, and coordinates.

Day 11: Impact Graph

- Build Neo4j traversal:
Change → Stale Assumption → Discipline → Activity → Commissioning

Day 12: Unified UI

- Before Construction dashboard
- During Construction dashboard
- Evidence/provenance viewer
- Agent status and investigation history

Day 13: Testing

- Test missing data, invalid models, API failures, bad extraction, retries, and cache fallback.
- Prepare 3 construction cases and 3–5 candidate sites.

Day 14: Final Demo

- Integrate and polish.
- Rehearse end-to-end workflow.
- No new features.